

Урок 10. Корутины. Context

18 февраля 2020

В этом уроке разбираемся с Context. Какие его элементы используются по умолчанию. Как создавать свой элемент. Как происходит передача элементов в контекст создаваемой корутины.

Перед тем, как обсуждать Context, давайте снова обратимся к теме Scope. Напомню, что scope нужен, чтобы запустить корутину. В scope есть Job, который будет являться родительским для всех корутин этого scope. Благодаря этому мы в любой момент можем отменить все корутины, просто отменив scope.

Но Job лежит в scope не сам по себе. Посмотрим на исходники CoroutineScope:

```
1 public interface CoroutineScope {
2     /**
3      * The context of this scope.
4      *
5      * By convention, should contain an instance of a [Job][Job] to enforce structured concurrency.
6      */
7     public val coroutineContext: CoroutineContext
8 }
```

В scope есть хранилище, называемое Context. И по контракту в нем должен быть Job. Как раз для того, чтобы быть родителем для корутин этого scope.

Context чем-то похож на Map. Он хранит в себе элементы, и их можно достать по ключу. В качестве примера получения элемента из context можно привести исходники метода cancel у scope:

```
1 public fun CoroutineScope.cancel(cause: CancellationException? = null) {
2     val job = coroutineContext[Job] ?: error("Scope cannot be cancelled because it does not have a job: $this")
3     job.cancel(cause)
4 }
```

Из контекста скопа здесь достается Job и вызывается его метод cancel.

Поместить в context можно только объект специального класса. Чуть позже в этом уроке мы научимся сами создавать такие классы.

Кроме Job в context чаще всего хранится диспетчер, который задает поток корутины. Чтобы сформировать Context, который будет содержать в себе Job и диспетчер (например, Dispatchers.Default), достаточно написать так:

```
1 val context = Job() + Dispatchers.Default
2 log("context = $context")
```

Это выражение создаст Context и поместит туда указанные элементы

Лог покажет содержимое контекста:

```
context = [JobImpl{Active}@42b0573, DefaultDispatcher]
```

В контексте лежат реализация джоба и диспетчер.

Этот сформированный контекст может быть использован при создании scope:

```
1 val context = Job() + Dispatchers.Default
2 val scope = CoroutineScope(context)
```

Теперь контекст созданного scope будет содержать в себе указанные Job и диспетчер.

Можно написать сразу так:

```
1 val scope = CoroutineScope(Job() + Dispatchers.Default)
```

Теперь давайте рассмотрим несколько особенностей работы контекста, и на примерах посмотрим, какой во всем этом смысл.

Контекст из одного элемента

Если помните, в прошлых уроках мы создавали scope следующим образом:

```
1 private val scope = CoroutineScope(Job())
```

Вместо Context мы передавали Job. Это работает, т.к. любой элемент, который можно поместить в context, сам по себе также является context-ом. Т.е. Job - это просто Context с одним элементом.

Аналогично и Dispatchers.Default - это Context с одним элементом. Можно сделать так:

```
1 val scope = CoroutineScope(Dispatchers.Default)
```

scope поймет, что ему передают контекст с одним элементом - диспетчером.

Job для scope будет создан, если его нет в контексте

Когда мы создаем scope и передаем ему контекст, выполняется проверка, что этот контекст содержит Job. И если не содержит, то Job будет создан.

Это можно увидеть в исходниках функции создания scope:

```
1 public fun CoroutineScope(context: CoroutineContext): CoroutineScope =
2     ContextScope(if (context[Job] != null) context else context + Job())
```

Выполняется проверка, есть ли Job в передаваемом контексте. Если есть, то контекст принимается. Иначе к нему добавляется новый Job.

Давайте убедимся в этом на примере. При создании scope укажем только диспетчер и выведем в лог итоговый контекст этого scope:

```
1 val scope = CoroutineScope(Dispatchers.Default)
2 log("context = ${scope.coroutineContext}")
```

Логи:

```
context = [JobImpl{Active}@42b0573, DefaultDispatcher]
```

Job был создан и помещен в контекст scope.

Как создать свой элемент для контекста

Допустим, у нас есть класс:

```
1 data class UserData(
2     val id: Long,
3     val name: String,
4     val age: Int
5 )
```

Чтобы иметь возможность поместить объект этого класса в контекст, необходимо наследовать AbstractCoroutineContextElement следующим образом:

```
1 data class UserData(
2     val id: Long,
3     val name: String,
4     val age: Int
5 ): AbstractCoroutineContextElement(UserData) {
6     companion object Key : CoroutineContext.Key<UserData>
7 }
```

Теперь такой объект можно помещать в контекст тем же способом, что и Job или диспетчер

```
1 val userData = UserData(1, "name1", 10)
2 val scope = CoroutineScope(Job() + Dispatchers.Default + userData)
```

А достать его из контекста можно так:

```
1 val userData = coroutineContext[UserData]
```

Зачем это может быть нужно? Например, чтобы сделать какие-либо данные доступными для всех корутин, включая вложенные, т.к. данные контекста передаются между корутинами. Давайте обсудим это подробно.

Передача данных контекста при создании корутин

В уроке про scope мы выяснили, что каждая корутина сама по себе является scope. Это необходимо, чтобы создавать дочерние корутины. Если быть более конкретным, то Job корутины является по совместительству ее скопом, а следовательно и содержит контекст корутины. Давайте разберемся, как этот контекст наполняется элементами.

Когда билдер создает новую корутину, он создает для нее новый пустой контекст и помещает туда элементы из контекста родителя этой корутины. Т.е. элементы контекста scope будут переданы в контекст корутины, созданной в этом scope. Аналогично элементы контекста родительской корутины будут переданы в контекст ее дочерней корутины.

Но есть пара нюансов.

Во-первых, Job не передается. Для создаваемой корутины создается новый Job, который подписывается на Job родителя и помещается в контекст созданной корутины.

Во-вторых, если при передаче выясняется, что отсутствует диспетчер, то будет взят диспетчер по умолчанию. Поэтому мы можем нигде явно не указывать диспетчер. В этом случае корутина сама возьмет себе дефолтный.

Давайте сделаем несколько примеров, чтобы увидеть, как происходит передача элементов контекста при создании корутин.

Напишем простой метод, чтобы доставать из контекста и выводить в лог Job и диспетчер. Это поможет нам наглядно понять, что происходит с контекстом.

```
1 private fun contextToString(context: CoroutineContext): String =
2     "Job = ${context[Job]}, Dispatcher = ${context[ContinuationInterceptor]}"
```

Для начала давайте проверим утверждения, которые я привел чуть выше. О том, что Job не передается, а диспетчер по умолчанию используется, если явно не указать диспетчер. А также познакомимся с пустым контекстом.

```
1 val scope = CoroutineScope(EmptyCoroutineContext)
2 log("scope, ${contextToString(scope.coroutineContext)}")
3
4 scope.launch {
5     log("coroutine, ${contextToString(coroutineContext)}")
6 }
```

EmptyCoroutineContext - это пустой контекст. Создаем с ним scope и запускаем корутину в этом scope. Посмотрим какие элементы будут в контексте скопа и какие из них будут переданы в контекст созданной корутины.

Логи:

```
scope, Job = JobImpl{Active}@b5c0c8d, Dispatcher = null
coroutine, Job = StandaloneCoroutine{Active}@7640e53, Dispatcher = DefaultDispatcher
```

Смотрим на контекст скопа.

Скоп не нашел Job в переданном ему пустом контексте, создал свой Job и поместил его в свой контекст. Потому что в scope всегда должен быть Job. Он будет выступать родителем для создаваемых корутин.

А вот диспетчера в scope может и не быть. Диспетчер отвечает за выбор потока для выполнения кода. Но scope используется для старта корутин. Сам по себе он не выполняет никакой код. Поэтому ему не нужен диспетчер. И т.к. мы никакой диспетчер ему не передавали, то scope остается без диспетчера в своем контексте.

Смотрим на контекст корутины.

Для нее был создан новый Job. Потому что у каждой корутины должен быть свой собственный Job, который отвечает за состояние корутины и ее результат.

В качестве диспетчера в контекст корутины был помещен диспетчер по умолчанию, потому что из контекста scope не пришел никакой диспетчер, а он необходим для выполнения кода корутины. Иначе корутине просто неоткуда будет взять поток для выполнения своего кода.

Как видите, в этом примере у нас вообще ничего не передалось из контекста скопа в контекст корутины. Были только автоматически созданы минимально необходимые для работы элементы.

Давайте поменяем пример, чтобы увидеть передачу. Мы знаем, что Job не передается. Но вот диспетчер, если он есть, будет передан. Давайте в scope укажем диспетчер и проверим, будет ли он передан в контекст корутины. Для чистоты эксперимента возьмем не дефолтный диспетчер, а Main. Корутина, использующая этот диспетчер, будет выполнена в main потоке.

Указываем Main диспетчер при создании scope:

```
1 val scope = CoroutineScope(Dispatchers.Main)
2 log("scope, ${contextToString(scope.coroutineContext)}")
3
4 scope.launch {
5     log("coroutine, ${contextToString(coroutineContext)}")
6 }
```

Смотрим лог

```
scope, Job = JobImpl{Active}@7640e53, Dispatcher = Main
coroutine, Job = StandaloneCoroutine{Active}@309e489, Dispatcher = Main
```

В контексте scope есть Main диспетчер. И он же есть в контексте корутины, потому что был передан из контекста скопа. Системе не пришлось помещать в корутину диспетчер по умолчанию, потому что корутина получила диспетчер из скопа.

Вложенные корутины

Создадим в корутине еще одну корутину, а в ней еще одну. Посмотрим, как будет передан диспетчер от родительских корутин к дочерним.

```
1 val scope = CoroutineScope(Job() + Dispatchers.Main)
2 log("scope, ${contextToString(scope.coroutineContext)}")
3
4 scope.launch {
5     log("coroutine, level1, ${contextToString(coroutineContext)}")
6
7     launch {
8         log("coroutine, level2, ${contextToString(coroutineContext)}")
9
10        launch {
11            log("coroutine, level3, ${contextToString(coroutineContext)}")
12        }
13    }
14 }
```

Логи:

```
scope, Job = JobImpl{Active}@973e4a6, Dispatcher = Main
coroutine, level1, Job = StandaloneCoroutine{Active}@4e813a2, Dispatcher = Main
coroutine, level2, Job = StandaloneCoroutine{Active}@e17e833, Dispatcher = Main
coroutine, level3, Job = StandaloneCoroutine{Active}@35f97f0, Dispatcher = Main
```

Диспетчер Main передается по цепочке: от scope к level1, от level1 к level2, от level2 к level3.

Контекст для корутины

Чтобы повлиять на содержимое контекста корутины, мы можем передать контекст в билдер этой корутины. Элементы этого контекста дополняют или заменяют элементы переданные из родительской корутины или скопа.

Например, мы хотим, чтобы корутина level1 работала с main диспетчером, а level2 и level3 - с диспетчером по умолчанию. Для этого мы должны передать Dispatchers.Default в билдер корутины level2. Тем самым мы заменим диспетчер, пришедший из level1 на свой диспетчер.

```
1  val scope = CoroutineScope(Job() + Dispatchers.Main)
2  log("scope, ${contextToString(scope.coroutineContext)}")
3
4  scope.launch {
5      log("coroutine, level1, ${contextToString(coroutineContext)}")
6
7      launch(Dispatchers.Default) {
8          log("coroutine, level2, ${contextToString(coroutineContext)}")
9
10         launch {
11             log("coroutine, level3, ${contextToString(coroutineContext)}")
12         }
13     }
14 }
```

Логи:

```
scope, Job = JobImpl{Active}@973e4a6, Dispatcher = Main
coroutine, level1, Job = StandaloneCoroutine{Active}@8fcba94, Dispatcher = Main
coroutine, level2, Job = StandaloneCoroutine{Active}@6ecb93d, Dispatcher = DefaultDispatcher
coroutine, level3, Job = StandaloneCoroutine{Active}@569a532, Dispatcher = DefaultDispatcher
```

В корутине level2 диспетчер меняется на DefaultDispatcher. И далее он уже будет передан в корутину level3.

Этот механизм передачи элементов контекста будет работать и с объектом UserData, который мы создавали ранее в этом уроке. Т.е. поместив данные пользователя в контекст скопа, вы сможете получить их из контекста в любой, в том числе и вложенной, корутине этого скопа. Попробуйте сделать это в качестве упражнения.